

Documentación técnica — Modelo de Roles de Datos e IA

Guía completa de lo que construimos en la Fase 1 y de cómo funciona el código, pensada para que puedas modificarlo a mano.

Parte 1 — Resumen de lo que hicimos (Fase 1)

Partiendo desde cero (sin Node.js instalado), llegamos a una aplicación web publicada en internet. Estos fueron los hitos:

1. **Instalación del entorno** — Node.js + npm en Windows.
2. **Creación del proyecto** — con Vite, usando la plantilla React + TypeScript.
3. **Estilos** — configuración de Tailwind CSS.
4. **La app** — un componente React que visualiza el modelo de roles del equipo.
5. **Control de versiones** — el código se subió a GitHub (repositorio `RolesDk`).
6. **Publicación** — primero en Vercel, luego migrado a GitHub Pages con despliegue automático.
7. **Mejoras de contenido** — rediseño con los colores corporativos de DataKnow, y un grafo más preciso con selector de tipo de proyecto.

Resultado: <https://josebarrita.github.io/RolesDk/>

Parte 2 — Cómo está organizado el proyecto

```
RolesDk/
├── .github/workflows/deploy.yml  # Publica la app automáticamente al hacer
push
├── public/                      # Archivos estáticos (favicon, etc.)
├── src/
│   ├── App.tsx                 ← AQUÍ ESTÁ TODO: datos, lógica y diseño
│   ├── main.tsx                # Punto de arranque (rara vez se toca)
│   └── index.css               # Una sola línea que activa Tailwind
├── index.html                  # HTML base (título de la pestaña, etc.)
├── vite.config.ts              # Configuración técnica (incluye la ruta de GitHub
Pages)
└── package.json                # Lista de dependencias y comandos
```

Lo importante: casi todo lo que querrás cambiar está en `src/App.tsx`. Los demás archivos son de configuración y casi no se tocan.

Parte 3 — Mapa de `src/App.tsx`

El archivo tiene ~709 líneas y se divide en cuatro grandes bloques. Aquí está la ubicación exacta de cada parte:

Bloque A — DATOS (líneas 1 a 265)

Todo el **contenido** (los textos que se ven en pantalla) vive aquí. Si quieres cambiar palabras, niveles o descripciones, es en esta zona — **sin tocar nada de programación**.

Qué es	Variable	Líneas	Para qué sirve
Orden de roles	<code>ROLE_ORDER</code>	5	El orden en que aparecen los 4 roles
Roles completos	<code>ROLES</code>	7-188	El corazón: nombre, color, misión, los 5 niveles y la progresión de cada rol
Nombres de niveles	<code>LEVELS</code>	190	Junior, Semisenior, Senior, Líder Técnico, Líder Especialista
Dimensiones	<code>DIMS</code>	191-197	Las 5 fichas de cada nivel (Responsabilidades, Habilidades, etc.)
Flujos del grafo	<code>VALUE_FLOW</code>	199-206	Las flechas entre roles y sus etiquetas
Tipos de proyecto	<code>PROJECT_TYPES</code>	210-217	El selector y qué flujos activa cada proyecto
Movimientos laterales	<code>LATERAL</code>	218-223	Las transiciones entre roles (modo "Movimientos laterales")
Agile Manager	<code>AGILE</code>	225-246	Los textos de la pestaña Agile Manager
Matriz RACI	<code>RACI</code>	248-263	Las filas de la tabla RACI
Columnas RACI	<code>RACI_COLS</code>	264	Los encabezados de la tabla
Colores RACI	<code>RACI_COLORS</code>	265	El color de cada letra (A/R, R, C, I)

Bloque B — HELPERS (líneas 267 a 273)

Qué es	Función	Líneas	Para qué sirve
Cálculo de flechas	<code>edge()</code>	268-273	Matemática que dibuja las flechas entre roles sin que se solapen con las tarjetas. No necesitas tocarla.

Bloque C — COMPONENTES (líneas 275 a 654)

Cada "pantalla" de la app es un componente. Aquí está la lógica + el diseño.

Componente	Líneas	Qué dibuja
<code>GraphView</code>	276-458	El grafo de roles con el selector de proyectos
<code>RoleDetail</code>	460-547	La vista de detalle de un rol (sus 5 niveles)
<code>AgileView</code>	549-597	La pestaña del Agile Manager
<code>RaciView</code>	599-654	La tabla de la matriz RACI

Bloque D — RAÍZ (líneas 656 al final)

Componente	Líneas	Qué hace
<code>App</code>	657-709	El componente principal. Arma el encabezado, las 3 pestañas y decide qué vista mostrar.

Parte 4 — Cómo entender la estructura de un ROL

Esta es la parte que más vas a editar. Cada rol (dentro de `ROLES`) tiene esta forma:

```
js
```

```

de: { // "de" = clave corta del rol (de, ds, bi, i
  name: "Ingeniero de Datos", // Nombre completo
  short: "Ing. Datos", // Nombre corto (para espacios pequeños)
  icon: Database, // Ícono (ver lista de íconos abajo)
  color: "#0a6fb8", // Color principal del rol
  soft: "rgba(10,111,184,0.10)", // Versión suave del color (fondos)
  cloud: "Compute + Storage...", // Analogía con la nube (texto largo)
  cloudShort: "Infraestructura base", // Analogía corta
  mission: "Garantizar que...", // La misión del rol
  pos: { x: 150, y: 310 }, // Posición en el grafo (coordenadas)

  levels: { // Los 5 niveles de seniority
    Junior: {
      resp: "...", // Responsabilidades
      hab: "...", // Habilidades
      alc: "...", // Alcance
      aut: "...", // Autonomía
      kpi: "..." // KPIs (indicadores)
    },
    Semisenior: { ... },
    // ... y así hasta Líder Especialista
  },

  progression: [ // Cómo se sube de nivel
    {
      to: "Semisenior", // Hacia qué nivel
      time: "1.5 - 2.5 años", // Cuánto suele tardar
      criteria: "...", // Qué debe cumplir
      change: "..." // El cambio de mentalidad
    },
    // ... una entrada por cada salto de nivel
  ],
}

```

Ejemplo: cambiar un texto

Si quieres cambiar la misión del Ingeniero de Datos, buscas la línea ~10 y editas el texto entre comillas de `mission:`. Eso es todo.

Esta es la parte con más lógica. Te explico la mecánica:

Los flujos (`VALUE_FLOW`), línea 199)

Cada flecha del grafo es un objeto con un `id` único:

```
js
{ id: "de-ia", a: "de", b: "ia", label: "datos para IA" }
//   ↑ identificador   ↑ desde   ↑ hacia   ↑ texto en la flecha
```

Los proyectos (`PROJECT_TYPES`), línea 210)

Cada tipo de proyecto dice **qué flujos están activos**:

```
js
{
  key: "rag",                      // identificador interno
  label: "RAG corporativo",        // lo que ve el usuario en el botón
  active: ["de-ia", "ia-bi"],      // flujos que se ven en color
  opt: ["ds-ia"]                  // flujos "opcionales" (punteados)
}
```

La regla es simple:

- Si un flujo está en `active` → se ve en color y resaltado.
- Si está en `opt` → se ve punteado con la etiqueta "(opcional)".
- Si no está en ninguno → se atenúa en gris.

Para agregar un nuevo tipo de proyecto

Solo añades un objeto nuevo a `PROJECT_TYPES`. Por ejemplo:

```
js
{ key: "vision", label: "Visión por computadora", active: ["de-ds", "ds-ia", "i
```

El botón aparece solo en la app, sin tocar nada más. Los `id` válidos para `active` y `opt` son los que definiste en `VALUE_FLOW`: `de-ds`, `de-ia`, `de-bi`, `ds-ia`, `ds-bi`, `ia-bi`.

Parte 6 — Referencia rápida para cambios comunes

Quiero cambiar...	Voy a...	Línea aprox.
El texto de un nivel de un rol	<code>ROLES</code> → el rol → <code>levels</code>	7-188
Los colores de un rol	<code>ROLES</code> → el rol → <code>color</code> y <code>soft</code>	(cada rol)
La posición de un rol en el grafo	<code>ROLES</code> → el rol → <code>pos</code>	(cada rol)
Agregar/quitar un tipo de proyecto	<code>PROJECT_TYPES</code>	210
Cambiar qué flujos activa un proyecto	<code>PROJECT_TYPES</code> → <code>active</code> / <code>opt</code>	210
El texto de una flecha del grafo	<code>VALUE_FLOW</code> → <code>label</code>	199
Una fila de la matriz RACI	<code>RACI</code>	248
El título de la app (pestaña del navegador)	<code>index.html</code>	(otro archivo)
El encabezado dentro de la app	<code>App</code> → busca el <code><h1></code>	~680

Parte 7 — Sobre los colores

La paleta corporativa de DataKnow que usamos:

Color	Código	Se usa en
Azul corporativo	<code>#0a6fb8</code>	Ingeniero de Datos / acentos principales
Cyan	<code>#1aa3c4</code>	Científico de Datos / degradados
Teal	<code>#0d9488</code>	Ingeniero de IA
Ámbar	<code>#f59e0b</code>	Analista BI
Gris azulado	<code>#94a3b8</code>	Textos secundarios / "Informed"

Los degradados de botones y encabezados usan `linear-gradient(135deg, #0a6fb8, #1aa3c4)`.

Parte 8 — Los íconos

La app usa la librería `lucide-react`. Los íconos disponibles ya importados (línea 2) son:

`Database`, `Brain`, `BarChart3`, `Sparkles`, `Cloud`, `Target`, `Wrench`, `ListChecks`, `Compass`, `Gauge`, `ArrowRight`, `ArrowLeft`, `Network`, `Table2`, `Workflow`, `ChevronRight`.

Para usar un ícono nuevo, primero lo agregas a esa lista de importación (línea 2) y luego ya puedes usarlo. El catálogo completo está en <https://lucide.dev/icons/>

Parte 9 — Flujo de trabajo para hacer cambios

Cada vez que quieras modificar algo:

```
bash

# 1. Edita src/App.tsx en VS Code y guarda

# 2. Pruébalo en local
npm run dev
# Abre http://localhost:5173/ y revisa que se vea bien

# 3. Si te gusta, publícalo
git add .
git commit -m "describe tu cambio aquí"
git push
```

Después del `push`, GitHub Pages republica la app sola en 1-2 minutos.

Consejo: siempre prueba con `npm run dev` antes de hacer `push`. Así detectas errores antes de publicar. Si la terminal muestra errores en rojo, la app tiene un problema que hay que corregir primero.

Parte 10 — Glosario para Fase 2

Términos que usamos y conviene tener claros:

- **Componente** — un bloque reutilizable de interfaz (como `GraphView`). Empieza con mayúscula.
- **Estado** (`useState`) — la "memoria" de un componente; por ejemplo, qué proyecto está seleccionado.
- **Props** — los datos que un componente recibe de otro (como `onSelect`).

- **Tailwind** — las clases entre comillas en `className` que dan estilo (`text-slate-800`, `rounded-xl`, etc.).
 - **SVG** — el formato con que se dibuja el grafo (líneas, flechas, formas).
 - **Deploy / despliegue** — el proceso de publicar la app en internet.
 - **Commit** — una "fotografía" guardada de tus cambios en Git.
-

Documento generado al cierre de la Fase 1. Para la Fase 2, esta guía te permite ubicar y modificar cualquier parte del código por tu cuenta.